Recorrido en profundidad: los tipos de arista

Soluciones

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Contenido

1 Los tipos de arista en un grafo dirigido

2 El mismo recorrido en un grafo no dirigido

3 Ciclos con amplitud: los grados de entrada

4 Recorrer sin usar caminos cerrados

Contenido

1 Los tipos de arista en un grafo dirigido
- El código
- Los bordes
- Lo que cuesta

Lo que devuelve `clasificar`

Entra la lista de adyacencia

Sale la lista de ternas (u, v, tipo) en el orden en que se miran, junto con los arreglos d y f. Con G_1: $(0, 1, \text{árbol})$, $(1, 2, \text{árbol})$, $(2, 0, \text{retroceso})$, $(0, 2, \text{avance})$, $(3, 1, \text{cruzada})$, $(3, 4, \text{árbol})$, con $d = [1, 2, 3, 7, 8]$ y $f = [6, 5, 4, 10, 9]$.

El código en Python: la regla

```
BLANCO = 0
GRIS = 1
NEGRO = 2


def tipo_de(color, d, u, v):
    # La regla: el estado de v y, si es negro, los tiempos.
    tipo = "cruzada"
    if color[v] == BLANCO:
        tipo = "arbol"
    elif color[v] == GRIS:
        tipo = "retroceso"
    elif d[u] < d[v]:
        tipo = "avance"
    return tipo
```

El código en Python: visitar

```
def visitar(adj, u, color, d, f, reloj, aristas):
    # Recorre desde u y anota el tipo de cada arista que mira.
    color[u] = GRIS
    reloj[0] = reloj[0] + 1
    d[u] = reloj[0]
    for v in adj[u]:
        aristas.append((u, v, tipo_de(color, d, u, v)))
        if color[v] == BLANCO:
            visitar(adj, v, color, d, f, reloj, aristas)
    color[u] = NEGRO
    reloj[0] = reloj[0] + 1
    f[u] = reloj[0]
```

El código en Python: un arranque por nodo blanco

```
def clasificar(adj):
    # Un arranque por cada nodo que siga en blanco.
    n = len(adj)
    color = [BLANCO] * n
    d = [0] * n
    f = [0] * n
    reloj = [0]
    aristas = []
    for u in range(n):
        if color[u] == BLANCO:
            visitar(adj, u, color, d, f, reloj, aristas)
    resultado = (aristas, d, f)
    return resultado
```

El código en C++: los datos

```cpp
#include <cstdio>
#include <string>
#include <vector>
using namespace std;

const int BLANCO = 0;
const int GRIS = 1;
const int NEGRO = 2;

struct Arista {
  int u;
  int v;
  string tipo;
};

struct Recorrido {
  vector<Arista> aristas;
  vector<int> d;
  vector<int> f;
};
```

El código en C++: la regla

```
string tipo_de(const vector<int> &color, const vector<int> &d,
               int u, int v) {
  // La regla: el estado de v y, si es negro, los tiempos.
  string tipo = "cruzada";
  if (color[v] == BLANCO) {
    tipo = "arbol";
  } else if (color[v] == GRIS) {
    tipo = "retroceso";
  } else if (d[u] < d[v]) {
    tipo = "avance";
  }
  return tipo;
}
```

El código en C++: visitar

```
void visitar(const vector<vector<int> > &adj, int u,
             vector<int> &color, int &reloj, Recorrido &r) {
  // Recorre desde u y anota el tipo de cada arista que mira.
  color[u] = GRIS;
  reloj = reloj + 1;
  r.d[u] = reloj;
  for (size_t i = 0; i < adj[u].size(); i++) {
    int v = adj[u][i];
    Arista a = {u, v, tipo_de(color, r.d, u, v)};
    r.aristas.push_back(a);
    if (color[v] == BLANCO) {
      visitar(adj, v, color, reloj, r);
    }
  }
  color[u] = NEGRO;
  reloj = reloj + 1;
  r.f[u] = reloj;
}
```

El código en C++: un arranque por nodo blanco

```
Recorrido clasificar(const vector<vector<int> > &adj) {
  // Un arranque por cada nodo que siga en blanco.
  int n = adj.size();
  vector<int> color(n, BLANCO);
  Recorrido r;
  r.d.assign(n, 0);
  r.f.assign(n, 0);
  int reloj = 0;
  for (int u = 0; u < n; u++) {
    if (color[u] == BLANCO) {
      visitar(adj, u, color, reloj, r);
    }
  }
  return r;
}
```

Los bordes

Caso	Sale
Grafo vacío	ninguna arista, d y f vacíos
Nodo aislado	su propio arranque: $u.f = u.d + 1$
Varias componentes	un arranque en cada una; entre árboles solo hay cruzadas
Arista de u a sí mismo	retroceso: u está gris al mirarla

Los bordes: la profundidad de las llamadas

Un camino largo

Cada nodo del camino abre una llamada. Medido sobre un camino de n nodos, con los valores por omisión, Python da `RecursionError` pasados 998 nodos. Para grafos así se sube el límite o se cambia la recursión por una pila propia.

Lo que cuesta

De qué depende

Cada nodo tiene una llamada, y en ella se recorre su lista una vez. Decidir el tipo es leer un color y dos tiempos. El ciclo de afuera mira V nodos:

$$\Theta(V + E).$$

En memoria van los tres arreglos, la lista de E ternas y la pila de llamadas.

Contenido

2 El mismo recorrido en un grafo no dirigido
- El código
- Los bordes
- Lo que cuesta

Lo que devuelve `clasificar_no_dirigido`

Una terna por cada vez que se mira una arista

Con G_2 salen diez, dos por arista: árbol, padre, árbol, retroceso, padre, árbol, padre, árbol, padre y repetida. Las de padre y repetida son la otra cara de una que ya se clasificó.

El código en Python: visitar con el padre

```
BLANCO = 0
GRIS = 1
NEGRO = 2


def visitar_nd(adj, u, padre, color, aristas):
    # Como en el dirigido, pero la arista hacia el padre se salta.
    color[u] = GRIS
    for v in adj[u]:
        if v == padre:
            aristas.append((u, v, "padre"))
        elif color[v] == BLANCO:
            aristas.append((u, v, "arbol"))
            visitar_nd(adj, v, u, color, aristas)
        elif color[v] == GRIS:
            aristas.append((u, v, "retroceso"))
        else:
            aristas.append((u, v, "repetida"))
    color[u] = NEGRO
```

El código en Python: un arranque por nodo blanco

```
def clasificar_no_dirigido(adj):
    # La raiz de cada arranque no tiene padre: se usa -1.
    n = len(adj)
    color = [BLANCO] * n
    aristas = []
    for u in range(n):
        if color[u] == BLANCO:
            visitar_nd(adj, u, -1, color, aristas)
    return aristas
```

El código en C++: visitar con el padre

```cpp
void visitar_nd(const vector<vector<int> > &adj, int u,
                int padre, vector<int> &color,
                vector<Arista> &aristas) {
  // Como en el dirigido, pero se salta la del padre.
  color[u] = GRIS;
  for (size_t i = 0; i < adj[u].size(); i++) {
    int v = adj[u][i];
    Arista a = {u, v, "repetida"};
    if (v == padre) {
      a.tipo = "padre";
    } else if (color[v] == BLANCO) {
      a.tipo = "arbol";
    } else if (color[v] == GRIS) {
      a.tipo = "retroceso";
    }
    aristas.push_back(a);
    if (v != padre && color[v] == BLANCO) {
      visitar_nd(adj, v, u, color, aristas);
    }
  }
  color[u] = NEGRO;
}
```

El código en C++: un arranque por nodo blanco

```cpp
vector<Arista> clasificar_no_dirigido(
    const vector<vector<int> > &adj) {
  // La raiz de cada arranque no tiene padre: se usa -1.
  int n = adj.size();
  vector<int> color(n, BLANCO);
  vector<Arista> aristas;
  for (int u = 0; u < n; u++) {
    if (color[u] == BLANCO) {
      visitar_nd(adj, u, -1, color, aristas);
    }
  }
  return aristas;
}
```

Los bordes

Caso	Sale
Grafo vacío o sin aristas	ninguna terna, sin ciclo
Varias componentes	un arranque en cada una, con padre -1
Arista de u a sí mismo	retroceso: $v = u$ está gris y no es el padre
Dos aristas entre u y v	las dos se saltan como padre, y el ciclo de dos se pierde

Los bordes: aristas repetidas

Saltar una sola vez

Si la entrada puede traer dos aristas entre los mismos nodos, el padre se salta solo la primera vez que aparece en la lista; la segunda ya cuenta como retroceso. Otra forma es pasar el número de la arista en lugar del nodo padre.

Lo que cuesta

De qué depende

Cada arista está en dos listas, así que se mira $2E$ veces. Cada nodo tiene una llamada:

$$\Theta(V + E).$$

Para saber solo si hay ciclo basta la primera de retroceso. Sin aristas repetidas, con $E \geq V$ ya hay ciclo.

Contenido

3 Ciclos con amplitud: los grados de entrada
- El código
- Los bordes
- Lo que cuesta

Lo que devuelve `tiene_ciclo_kahn`

Si hay ciclo y quiénes no salen

En Python, el par (hay ciclo, los que quedan); en C++, un booleano, y los que quedan en un vector por referencia. Con G_1: cierto y $[0, 1, 2]$. Sin la arista $2 \to 0$: falso y la lista vacía.

El código en Python: los grados de entrada

```
from collections import deque


def grados_de_entrada(adj):
    # Cuantas aristas llegan a cada nodo.
    grado = [0] * len(adj)
    for u in range(len(adj)):
        for v in adj[u]:
            grado[v] = grado[v] + 1
    return grado
```

El código en Python: los que quedan y los de grado cero

```
def sin_salir(salio):
    # Los nodos que la cola nunca entrego.
    quedan = []
    for u in range(len(salio)):
        if not salio[u]:
            quedan.append(u)
    return quedan


def cola_de_ceros(grado):
    # Los que no tienen aristas que les lleguen.
    cola = deque()
    for u in range(len(grado)):
        if grado[u] == 0:
            cola.append(u)
    return cola
```

El código en Python: la cola

```
def tiene_ciclo_kahn(adj):
    # Devuelve si hay ciclo y los nodos que nunca salen de la cola.
    grado = grados_de_entrada(adj)
    cola = cola_de_ceros(grado)
    salio = [False] * len(adj)
    while len(cola) > 0:
        u = cola.popleft()
        salio[u] = True
        for v in adj[u]:
            grado[v] = grado[v] - 1
            if grado[v] == 0:
                cola.append(v)
    quedan = sin_salir(salio)
    resultado = (len(quedan) > 0, quedan)
    return resultado
```

El código en C++: los grados de entrada

```cpp
#include <cstdio>
#include <queue>
#include <vector>
using namespace std;

vector<int> grados_de_entrada(const vector<vector<int> > &adj) {
  // Cuantas aristas llegan a cada nodo.
  vector<int> grado(adj.size(), 0);
  for (size_t u = 0; u < adj.size(); u++) {
    for (size_t i = 0; i < adj[u].size(); i++) {
      grado[adj[u][i]] = grado[adj[u][i]] + 1;
    }
  }
  return grado;
}
```

El código en C++: los que quedan y los de grado cero

```cpp
vector<int> sin_salir(const vector<bool> &salio) {
  // Los nodos que la cola nunca entrego.
  vector<int> quedan;
  for (size_t u = 0; u < salio.size(); u++) {
    if (!salio[u]) {
      quedan.push_back(u);
    }
  }
  return quedan;
}

queue<int> cola_de_ceros(const vector<int> &grado) {
  // Los que no tienen aristas que les lleguen.
  queue<int> cola;
  for (size_t u = 0; u < grado.size(); u++) {
    if (grado[u] == 0) {
      cola.push(u);
    }
  }
  return cola;
}
```

El código en C++: la cola

```cpp
bool tiene_ciclo_kahn(const vector<vector<int> > &adj,
                      vector<int> &quedan) {
  // Devuelve si hay ciclo; en quedan, los que nunca salen.
  vector<int> grado = grados_de_entrada(adj);
  queue<int> cola = cola_de_ceros(grado);
  vector<bool> salio(adj.size(), false);
  while (!cola.empty()) {
    int u = cola.front();
    cola.pop();
    salio[u] = true;
    for (size_t i = 0; i < adj[u].size(); i++) {
      int v = adj[u][i];
      grado[v] = grado[v] - 1;
      if (grado[v] == 0) {
        cola.push(v);
      }
    }
  }
  quedan = sin_salir(salio);
  return !quedan.empty();
}
```

Los bordes

Caso	Sale
Grafo vacío	sin ciclo, nadie queda
Nodo aislado	grado cero: entra de primero y sale
Varias componentes	todos los de grado cero entran al comienzo
Arista de u a sí mismo	su grado nunca baja de uno: u queda, y hay ciclo
Nodo que solo cuelga de un ciclo	queda, aunque no esté en el ciclo

Lo que cuesta

De qué depende

Contar los grados recorre las listas una vez. Cada nodo entra a la cola a lo sumo una vez, y al salir recorre su lista. Encontrar los que quedan mira los V nodos:

$$\Theta(V + E).$$

No hay recursión: un camino largo no llena la pila.

Contenido

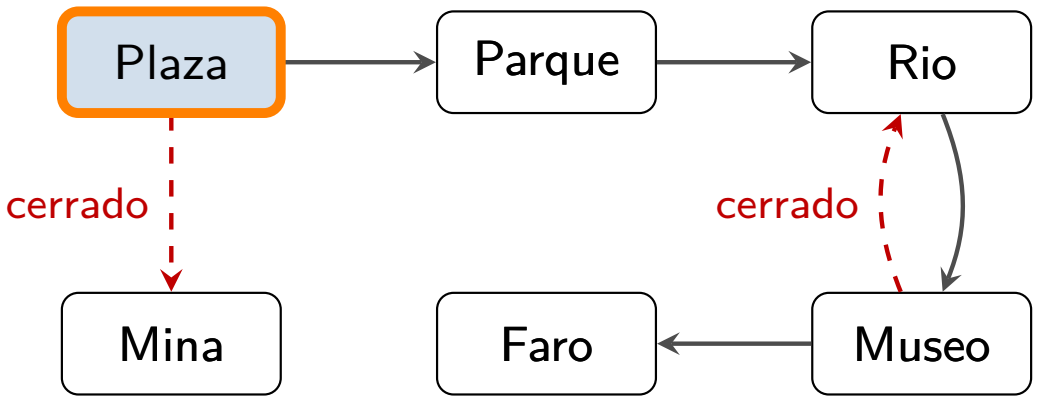
4 Recorrer sin usar caminos cerrados
- El modelo
- La traza
- El código
- Los bordes
- Lo que cuesta

El modelo: nodos, aristas y lo prohibido

Un recorrido con una pregunta más

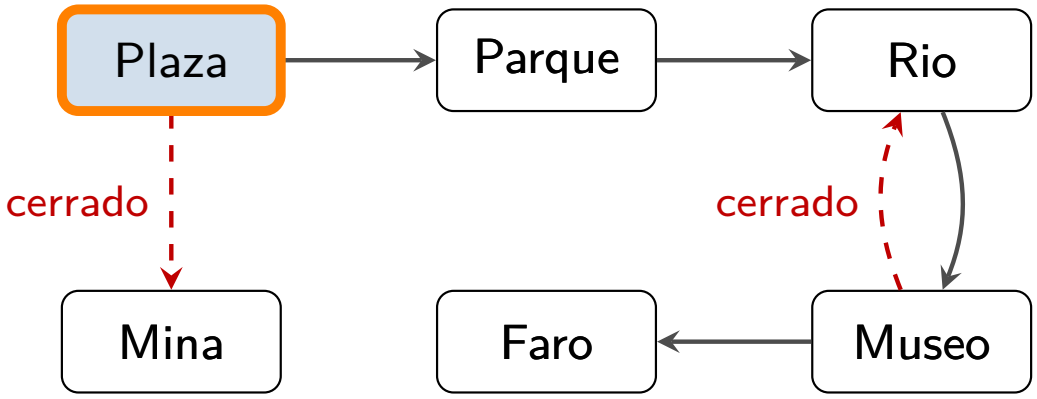
Los lugares son nodos y los caminos, aristas dirigidas. Los cerrados van a un conjunto de pares. La profundidad toma $u \to v$ si el par (u, v) no está en el conjunto y v no se ha visto. Lo que se descubre se anota en orden.

La traza desde Plaza



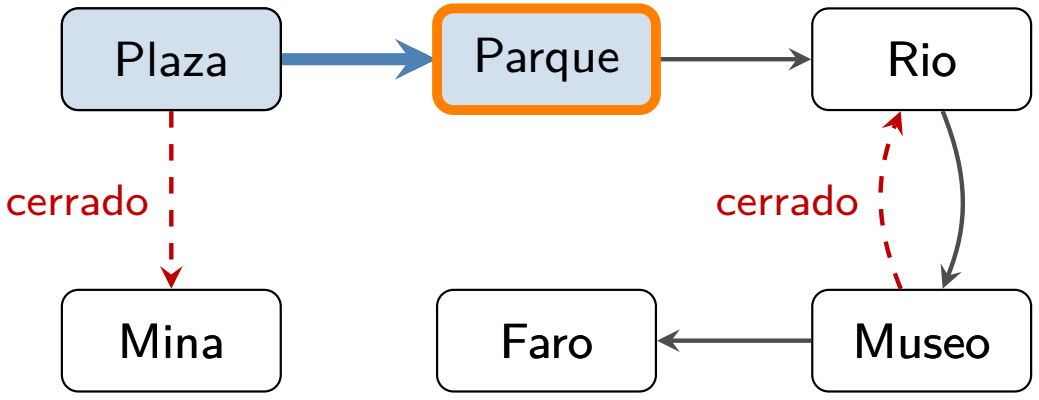
Entra Plaza: se marca visto y se anota. Orden: [Plaza].

La traza desde Plaza



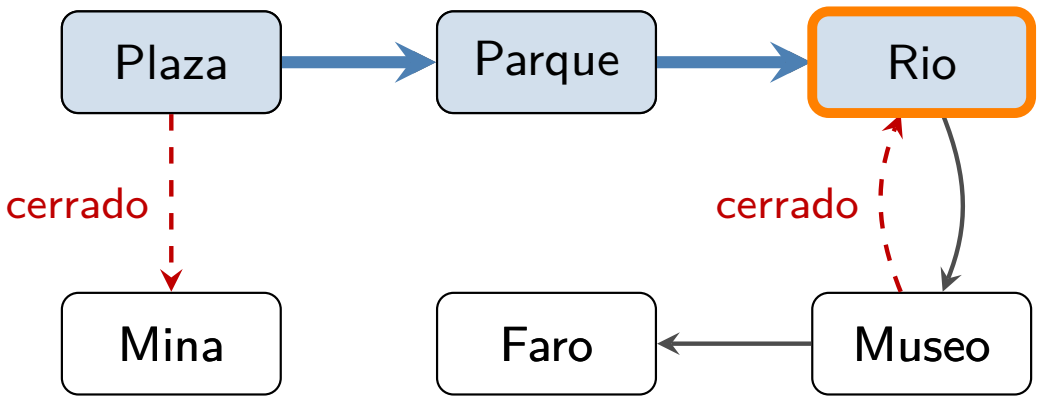
Plaza mira a Mina: el par (Plaza, Mina) está en el conjunto. No se toma.

La traza desde Plaza



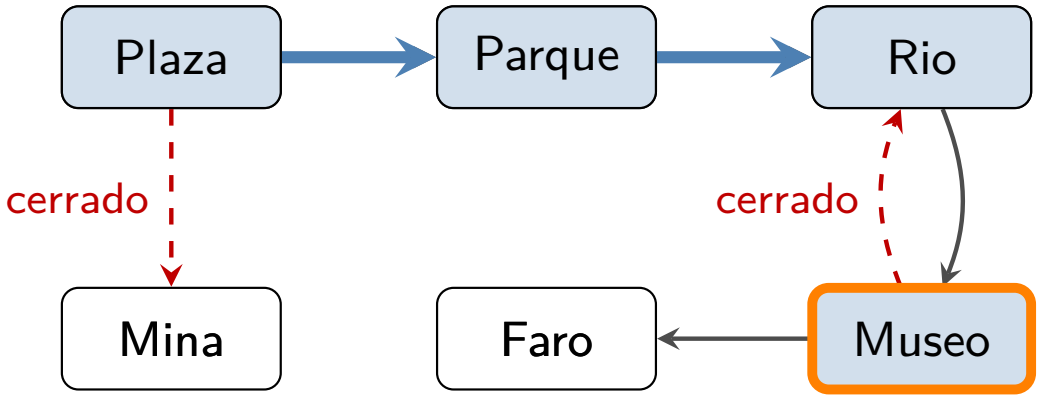
Plaza mira a Parque: abierto y sin ver. Se baja. Orden: [Plaza, Parque].

La traza desde Plaza



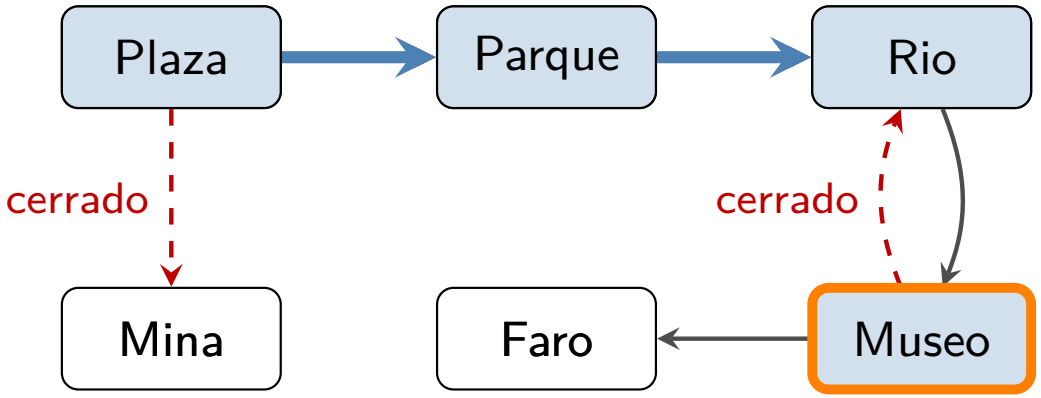
Parque mira a Rio: se baja. Orden: [Plaza, Parque, Rio].

La traza desde Plaza



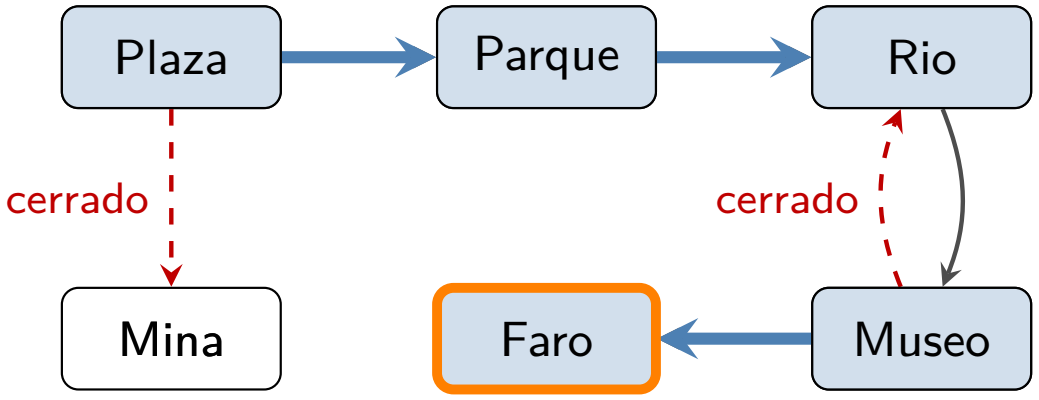
Rio mira a Museo: el par (Rio, Museo) no está en el conjunto. Se baja.
Orden: [Plaza, Parque, Rio, Museo].

La traza desde Plaza



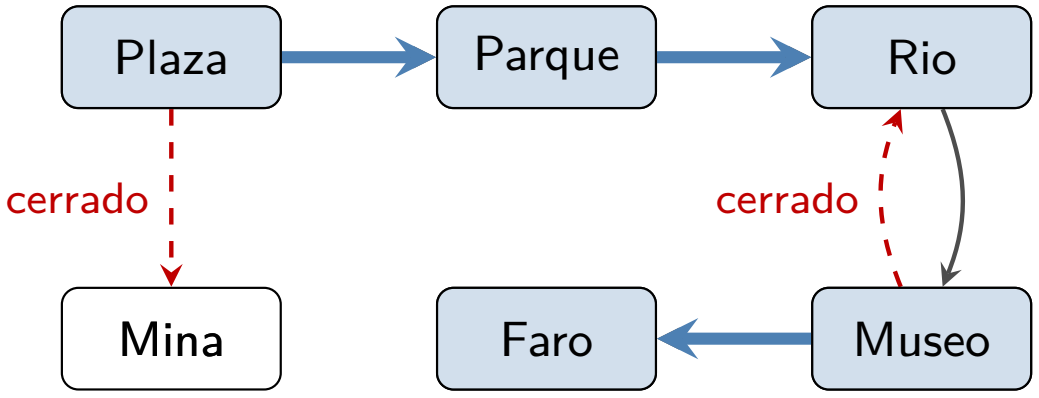
Museo mira a Rio: el par (Museo, Rio) está cerrado. No se toma, y Rio ya estaba visto.

La traza desde Plaza



Museo mira a Faro: se baja. Faro no tiene salidas y devuelve.

La traza desde Plaza



Las llamadas devuelven hasta Plaza. Sale [Plaza, Parque, Rio, Museo, Faro], y Mina queda afuera.

El código en Python: armar la lista

```
def armar(lugares, caminos):
    # Lista de adyacencia indexada por el nombre del lugar.
    adj = {}
    for lugar in lugares:
        adj[lugar] = []
    for origen, destino in caminos:
        adj[origen].append(destino)
    return adj
```

El código en Python: visitar sin los cerrados

```
def visitar(adj, prohibidos, u, visto, orden):
    # Profundidad que no toma un camino si su par esta cerrado.
    visto.add(u)
    orden.append(u)
    for v in adj[u]:
        if (u, v) not in prohibidos and v not in visto:
            visitar(adj, prohibidos, v, visto, orden)
```

El código en Python: `alcanzables`

```
def alcanzables(lugares, caminos, cerrados, inicio):
    # Los lugares a los que se llega, en orden de descubrimiento.
    adj = armar(lugares, caminos)
    prohibidos = set()
    for origen, destino in cerrados:
        prohibidos.add((origen, destino))
    visto = set()
    orden = []
    visitar(adj, prohibidos, inicio, visto, orden)
    return orden
```

El código en C++: armar la lista

```cpp
typedef pair<string, string> Camino;

typedef map<string, vector<string> > Mapa;

Mapa armar(const vector<string> &lugares,
           const vector<Camino> &caminos) {
  // Lista de adyacencia indexada por el nombre del lugar.
  Mapa adj;
  for (size_t i = 0; i < lugares.size(); i++) {
    adj[lugares[i]] = vector<string>();
  }
  for (size_t i = 0; i < caminos.size(); i++) {
    adj[caminos[i].first].push_back(caminos[i].second);
  }
  return adj;
}
```

El código en C++: visitar sin los cerrados

```
void visitar(Mapa &adj,
             const set<Camino> &prohibidos, const string &u,
             set<string> &visto, vector<string> &orden) {
  // Profundidad que no toma un camino si su par esta cerrado.
  visto.insert(u);
  orden.push_back(u);
  for (size_t i = 0; i < adj[u].size(); i++) {
    string v = adj[u][i];
    bool abierto = prohibidos.count(Camino(u, v)) == 0;
    if (abierto && visto.count(v) == 0) {
      visitar(adj, prohibidos, v, visto, orden);
    }
  }
}
```

El código en C++: alcanzables

```
vector<string> alcanzables(const vector<string> &lugares,
                           const vector<Camino> &caminos,
                           const vector<Camino> &cerrados,
                           const string &inicio) {
  // Los lugares a los que se llega, en orden de descubrimiento.
  Mapa adj = armar(lugares, caminos);
  set<Camino> prohibidos;
  for (size_t i = 0; i < cerrados.size(); i++) {
    prohibidos.insert(cerrados[i]);
  }
  set<string> visto;
  vector<string> orden;
  visitar(adj, prohibidos, inicio, visto, orden);
  return orden;
}
```

Los bordes

Caso	Sale
Inicio sin salidas	solo el inicio: [Faro]
Cerrado que no está entre los caminos	nada cambia: ese par nunca se pregunta
Un solo lugar, sin caminos	[inicio]
Dos caminos hacia el mismo lugar	aparece una vez, por el conjunto de vistos
Los dos sentidos cerrados	solo si los dos pares están en la lista

Lo que cuesta

De qué depende

Armar la lista cuesta $V + E$, y el conjunto de cerrados, C. El recorrido visita cada lugar una vez y mira cada camino una vez, con una consulta al conjunto. En Python, en promedio:

$$\Theta(V + E + C).$$

En C++, cada consulta a `map` o a `set` cuesta un logaritmo, y el total es $O((V + E + C) \log(V + C))$.

Referencias

- T. Cormen, C. Leiserson, R. Rivest y C. Stein. *Introduction to Algorithms*, tercera edición, 2009. Sección 22.3: recorrido en profundidad, tiempos y clasificación de aristas. Sección 22.4: orden topológico.